Report di valutazione della sicurezza

DVWA – SQL Injection e File Upload

Applicazione	Damn Vulnerable Web Application (DVWA)
Host analizzato	192.168.50.156
Ambiente	Laboratorio isolato, uso didattico
Strumenti	Browser web, Burp Suite (proxy), Kali Linux
Data	11/11/2026
Esito complessivo	2 vulnerabilità confermate: 1 critica, 1 alta

1. Sintesi esecutiva

Durante la verifica sull'ambiente DVWA sono state confermate due vulnerabilità. La prima è un'iniezione SQL nel modulo di ricerca per User ID, che permette di estrarre l'intero elenco degli utenti. La seconda, più grave, riguarda il modulo di caricamento file: il server accetta un file con estensione eseguibile lato server, lo salva in una directory raggiungibile dal web e lo esegue quando viene richiamato, restituendo l'output del comando di sistema.

La vulnerabilità di caricamento file consente l'esecuzione di comandi sul server. Va quindi corretta per prima.

2. Ambito e metodologia

La verifica è stata condotta sull'istanza DVWA del laboratorio, raggiungibile all'indirizzo 192.168.50.156. Le richieste sono state intercettate con Burp Suite per analizzare parametri e risposte. Le schermate riportate nel documento mostrano i risultati ottenuti. Il codice dello script utilizzato per la dimostrazione non è riportato: la schermata del terminale che lo visualizza è stata esclusa, e il documento descrive il comportamento osservato e i rimedi.

3. Riepilogo dei rilievi

#	Vulnerabilità	Gravità	Modulo DVWA
1	Unrestricted File Upload con esecuzione di comandi	Critica	File Upload
2	SQL Injection nella ricerca per User ID	Alta	SQL Injection

4. Dettaglio dei rilievi

4.1 Unrestricted File Upload con esecuzione di comandi (Critica)

Descrizione

Il modulo di caricamento è pensato per le immagini, ma non verifica in modo affidabile il tipo reale del file né l'estensione. Il file viene salvato con il nome originale in una directory pubblica (/hackable/uploads/).

Evidenze osservate

- Il server ha confermato il caricamento di un file con estensione .php, pur trattandosi di un modulo destinato alle immagini (Figura 2).
- Il file caricato è stato richiamato direttamente dal browser. Il server ha eseguito il contenuto come script e ha restituito l'output di un comando di sistema, con l'elenco dei file della directory (Figura 3).
- La richiesta è stata analizzata con Burp Suite, che conferma il passaggio dei parametri dal client al server.

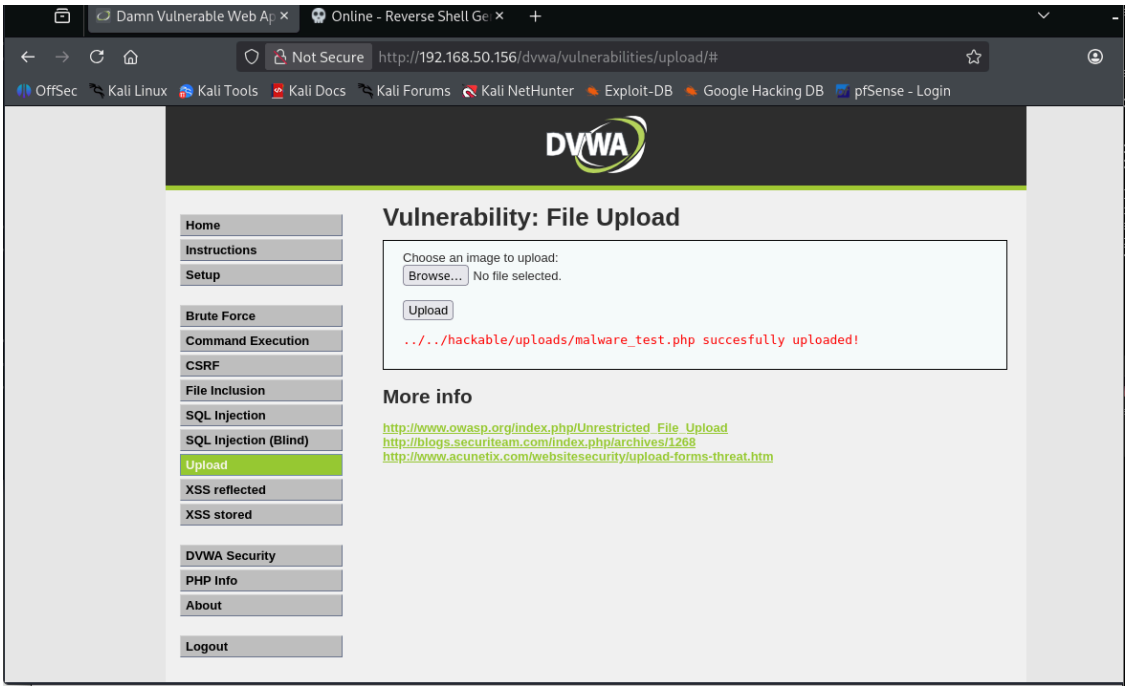


Figura 2 – Modulo di caricamento: il file .php viene accettato e salvato in /hackable/uploads/.

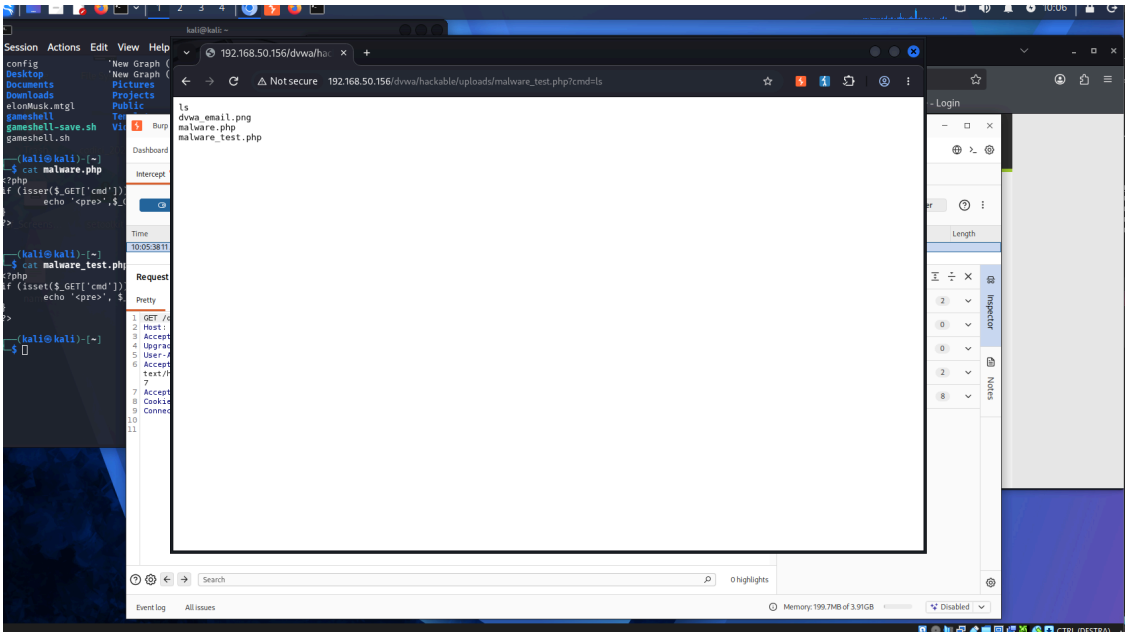


Figura 3 – Il file caricato viene richiamato dal browser e restituisce l'output del comando di sistema.

Impatto

Un attaccante può eseguire comandi arbitrari sul server con i privilegi dell'utente del servizio web. Questo permette di leggere e modificare file, accedere a dati sensibili e, se la rete è mal segmentata, spostarsi verso altri sistemi.

Rimedi raccomandati

- Accettare solo un elenco chiuso di estensioni per le immagini (ad esempio .jpg, .png, .gif), verificando il lato server e non solo quello del browser.
- Controllare il contenuto reale del file (firma binaria) e rielaborare le immagini con una libreria di image processing, così che eventuale codice incorporato venga rimosso.
- Rinominare i file caricati con un nome casuale generato dal server e non usare il nome originale.
- Salvare i file fuori dalla webroot oppure servirli tramite un gestore che non li esegue.
- Disabilitare l'esecuzione di script nella directory di upload (configurazione di Apache o Nginx).
- Limitare dimensione massima e frequenza dei caricamenti, e scansionare i file con un antimalware.
- Eseguire il servizio web con un utente a privilegi minimi.

4.2 SQL Injection nella ricerca per User ID (Alta)

Descrizione

Il campo User ID non valida l'input e lo inserisce direttamente nella query SQL. Inserendo una stringa che rende la condizione sempre vera, la query restituisce tutti i record invece di uno solo.

Evidenze osservate

- Con l'input 1' OR '1' = '1 la pagina ha restituito cinque record: admin, Gordon Brown, Hack Me, Pablo Picasso e Bob Smith (Figura 1).
- L'applicazione mostra direttamente i dati della query nella risposta, senza filtri.

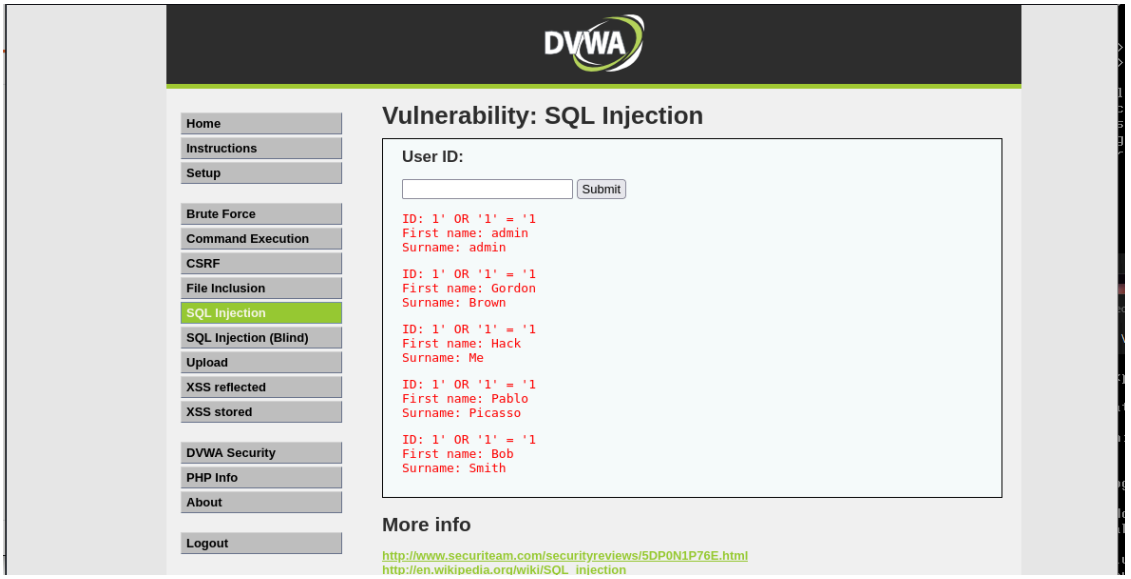


Figura 1 – Ricerca per User ID con l'input 1' OR '1' = '1: la pagina restituisce tutti i record.

Impatto

Divulgazione non autorizzata dei dati degli utenti. A seconda del database e dei privilegi, la vulnerabilità può permettere di leggere altre tabelle, aggirare l'autenticazione o modificare i dati.

Rimedi raccomandati

- Usare query parametrizzate (prepared statement) in ogni accesso al database, senza concatenare l'input utente.
- Validare il formato dell'input: per un ID numerico, accettare solo cifre e rifiutare il resto.
- Non mostrare messaggi di errore del database all'utente.
- Assegnare all'applicazione un account di database con solo i permessi necessari.
- Integrare test automatici (SAST e DAST) nel ciclo di sviluppo per intercettare questi errori prima della produzione.

5. Priorità di intervento

1. Correggere il caricamento file: impedisce l'esecuzione di codice sul server e va risolto subito.

2. Correggere le query SQL: adottare le query parametrizzate in tutta l'applicazione, non solo nel modulo di ricerca.

3. Verificare dopo le correzioni: ripetere i test per confermare che entrambe le vulnerabilità non siano più sfruttabili.

6. Riferimenti

- OWASP, Unrestricted File Upload: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- OWASP, SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

7. Conclusione

L'ambiente presenta due vulnerabilità che, combinate, permettono a un attaccante di ottenere il controllo del server. Le correzioni proposte sono standard e a basso costo per il loro effetto. Consiglio di applicarle e di ripetere la verifica al termine.